Axes (Not the Chopping Kind)

So far so completely clear and understandable, huh? Don't worry, this will soon make sense with some examples. First of all, let's look at axes ("ax-ees" – the plural of axis). There are thirteen possible axes in the XPath specification (don't panic at `namespace::`, we'll be covering XML/HTML namespaces in due course):

Table 11-11. *XPath axes*

`child::`	All immediate subordinate (one level down) nodes
`descendant::`	All subordinate nodes (at any depth)
`descendant-or-self::`	All descendant nodes and the current node
`parent::`	All immediate superordinate (one level up) nodes
`ancestor::`	All superordinate nodes (at any depth)
`ancestor-or-self::`	All ancestor nodes and the current node
`attribute::`	The attributes of nodes matching the current path
`following::`	All subsequent nodes except descendants
`following-sibling::`	All subsequent nodes from current (with the same parent)
`preceding::`	All preceding nodes except ancestors
`preceding-sibling::`	All preceding nodes from current (with the same parent)
`self::`	The current node (useful in certain contexts)
`namespace::`	All nodes in the given namespace

The axis component of the localization step can be written in the long form, with a `::` character sequence to delimit it from the node test. Some of you might be wondering at this stage why the XPath expression that we used – `'//*[@id="..."]'` – doesn't have any `::` in it. The reason is *abbreviations*.

Abbreviations

Common axis names have some handy shortcuts in order to make the expression syntax more concise. The default axis is `child::`; you can omit the axis name entirely, and the XPath engine will assume you mean the child (which makes sense; if you're used to